Complete Application and API Guide

This guide helps both technical and non-technical users understand, run, and use the local Order Resolution MCP application.

---

1) What this application does

The application is a local Order Resolution API for e-commerce support scenarios.

It:

- accepts JSON-RPC 2.0 requests through `/mcp/message`
- streams events through `/mcp/sse`
- fetches order data from an in-memory H2 database
- retrieves policy context from in-memory vector search (RAG)
- generates concise resolution suggestions from a local Ollama model
- enforces strict local execution with controlled LLM concurrency

---

2) Main use case (business view)

Support teams receive customer issues such as:

- "Where is my order?"
- "Can I cancel this order?"
- "Payment failed - what should I do?"

Agent/operator enters an `orderId` (example: `10002`), and the API returns:

- order status
- relevant policy
- recommended resolution text

No cloud AI dependency is required.

---

3) System architecture (simple)

Request flow:

1. Client sends JSON-RPC request to `/mcp/message`.
2. API validates request format and params (`orderId` regex).
3. API reads order from H2 (`OrderDatabaseTool`).
4. API searches policy snippets (`PolicyRagTool`).
5. API generates final answer via local Ollama (`AiConfig` service).
6. API returns strict JSON-RPC response.
7. Same result is emitted to SSE stream subscribers.

Core components:

- `AiConfig` - Ollama model config + semaphore lock
- `OrderDatabaseTool` - H2 table + Day-0 auto-seeding
- `PolicyRagTool` - vector store + policy ingestion
- `McpController` - JSON-RPC endpoint + SSE + circuit breaker

---

4) Learning path for new team members

Recommended order:

1. Read `README.md` (quick start).
2. Open Swagger UI (`/swagger-ui.html`) to view endpoints.
3. Run `tools/list` request.
4. Run `tools/call` with valid order IDs.
5. Watch `/mcp/sse` stream in parallel.
6. Read source files in this order:
    - `McpController`
    - `OrderDatabaseTool`
    - `PolicyRagTool`
    - `AiConfig`
7. Test error scenarios (invalid JSON-RPC, invalid orderId, unknown order).

---

5) Prerequisites

- macOS / Linux / Windows
- Java 21
- Maven 3.9+
- Ollama installed locally
- Enough local space for model files (keep only required models)

Required models:

- `phi3`
- `nomic-embed-text`

---

6) Installation and startup steps

Step A - Start Ollama

```
ollama serve
```

Step B - Pull models (one time)

```
ollama pull phi3
ollama pull nomic-embed-text
```

Step C - Start application

```
mvn spring-boot:run
```

Server URL:

- `http://localhost:8080`

---

7) Verification checklist (Day-0)

After startup:

- App starts without errors
- Logs show orders seeded
- Swagger UI opens
- `tools/list` returns one tool (`resolve_order`)
- `tools/call` works for `10001` to `10005`
- SSE stream receives heartbeat every 15 seconds

---

8) API details

8.1 Endpoint: `/mcp/message`

Method:

- `POST`

Content-Type:

- `application/json`

Protocol:

- JSON-RPC 2.0

Supported methods:

- `tools/list`
- `tools/call`

8.2 Endpoint: `/mcp/sse`

Method:

- `POST`

Content-Type:

- `text/event-stream`

Behavior:

- Streams JSON-RPC payloads as events
- Emits heartbeat comment every 15 seconds

---

9) JSON-RPC request/response examples

Example A - List tools

Request:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "tools/list",
  "params": {}
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "result": {
    "tools": [
      {
        "name": "resolve_order",
        "description": "Resolve order issues by orderId"
      }
    ]
  }
}
```

Example B - Resolve order

Request:

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "method": "tools/call",
```

```
  "params": {
    "orderId": "10002"
  }
}
```

Response shape:

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "result": {
    "orderId": "10002",
    "status": "SHIPPED",
    "policy": "SHIPPED: ...",
    "resolution": "..."
  }
}
```

Example C - Invalid orderId

Request:

```
{
  "jsonrpc": "2.0",
  "id": "3",
  "method": "tools/call",
  "params": {
    "orderId": "ABC"
  }
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": "3",
  "error": {
    "code": -32602,
    "message": "Invalid params: orderId must match ^[0-9]{5}$"
  }
}
```

---

10) Curl commands (copy/paste)

Stream SSE

```
curl -N -X POST http://localhost:8080/mcp/sse
```

Call tools/list

```
curl -s -X POST http://localhost:8080/mcp/message \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc":"2.0",
    "id":"1",
    "method":"tools/list",
    "params":{}
  }'
```

Call tools/call

```
curl -s -X POST http://localhost:8080/mcp/message \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc":"2.0",
    "id":"2",
    "method":"tools/call",
    "params":{"orderId":"10001"}
  }'
```

---

11) Non-technical end-user operating guide

This section is for users who are not developers.

Option 1 - Use Swagger UI (easiest)

1. Open browser:
   - `http://localhost:8080/swagger-ui.html`
2. Expand endpoint:
   - `POST /mcp/message`
3. Click **Try it out**.
4. Paste this request body:

```
{
  "jsonrpc": "2.0",
  "id": "ticket-1",
  "method": "tools/call",
  "params": {
    "orderId": "10003"
```

```
    }
  }
```

5. Click **Execute**.
6. Read:
   - `status`
   - `policy`
   - `resolution`

Option 2 - Use Postman (GUI tool)

1. Create request:
   - Method: `POST`
   - URL: `http://localhost:8080/mcp/message`
2. Header:
   - `Content-Type: application/json`
3. Body:
   - raw JSON (same as above)
4. Send request and read output.

End-user rules

- Use only 5-digit order IDs (`10001` format).
- If "Order not found", verify ID with operations team.
- If service unavailable, retry after 15-30 seconds.

---

12) Use case examples

Use case 1 - Customer asks to cancel order

Input:

- `orderId = 10001` (PROCESSING)

Expected behavior:

- API finds order in PROCESSING
- policy suggests cancellation window logic
- resolution returns concise action guidance

Use case 2 - Customer asks cancellation after shipping

Input:

- `orderId = 10002` (SHIPPED)

Expected behavior:

- API states shipped orders cannot be cancelled

- suggests return workflow after delivery

Use case 3 - Payment failed support call

Input:

- `orderId = 10004` (PAYMENT_FAILED)

Expected behavior:

- API suggests payment retry / alternate payment method

---

13) Default seeded orders

Auto-seeded on startup in H2 memory:

- `10001` - PROCESSING
- `10002` - SHIPPED
- `10003` - DELIVERED
- `10004` - PAYMENT_FAILED
- `10005` - CANCELLED

---

14) Troubleshooting

App fails to answer due to model error

Check:

- Ollama service is running
- both models are downloaded
- no typo in model names

API returns service unavailable

Cause:

- circuit breaker open due to recent failures

Action:

- wait for open-state duration and retry

SSE has no data

Check:

- client connected to `POST /mcp/sse`
- invoke `/mcp/message` in another terminal/window

---

15) Security and operational notes

- LLM execution is local-only (`localhost:11434`).
- Inference concurrency is restricted to one request at a time.
- JSON-RPC schema checks reject malformed input.
- Policy embeddings are in-memory and cleared on shutdown.

---

16) Quick reference

- Base URL: `http://localhost:8080`
- Swagger: `/swagger-ui.html`
- OpenAPI: `/v3/api-docs`
- JSON-RPC endpoint: `/mcp/message`
- SSE endpoint: `/mcp/sse`
- JSON-RPC version: `2.0`